

Grafika komputerowa

Dokumentacja do gry

Wykonali:

Konrad Zegar

Rzeszów 28.01.2024

- Instrukcja III

Według poleceń z instrukcji złożyliśmy obiekt (łazik) z 26 brył w programie Blender i zaimportowaliśmy go do naszego projektu.

```
6
7 bool loadOBJ(
8     const char* path,
9     std::vector<glm::vec3>& out_vertices,
10    std::vector<glm::vec2>& out_uv,
11    std::vector<glm::vec3>& out_normals,
12    std::vector<Material>& out_materials,
13    const char* Mtlpath
14) {
15    std::map<std::string, Material> materials;
16
17    printf("Loading OBJ file %s...\n", path);
18
19    std::vector<glm::vec3> temp_vertices;
20    std::vector<glm::vec2> temp_uv;
21    std::vector<glm::vec3> temp_normals;
22    std::string currentMaterial;
23
24    FILE* file = fopen(path, "r");
25    if (file == NULL) {
26        printf("Impossible to open the file ! Are you in the right path? See Tutorial 1 for details\n");
27        getchar();
28        return false;
29    }
30
31    materials = loadMTL(Mtlpath);
32
33    while (1) {
34        char lineHeader[128];
35        int res = fscanf(file, "%s", lineHeader);
36        if (res == EOF)
37            break;
38
39        if (strcmp(lineHeader, "v") == 0) {
40            glm::vec3 vertex;
41            fscanf(file, "%f %f %f\n", &vertex.x, &vertex.y, &vertex.z);
42            temp_vertices.push_back(vertex);
43        }
```

```

44     else if (strcmp(lineHeader, "vt") == 0) {
45         glm::vec2 uv;
46         fscanf(file, "%f %f\n", &uv.x, &uv.y);
47         uv.y = -uv.y;
48         temp_uvns.push_back(uv);
49     }
50     else if (strcmp(lineHeader, "vn") == 0) {
51         glm::vec3 normal;
52         fscanf(file, "%f %f %f\n", &normal.x, &normal.y, &normal.z);
53         temp_normals.push_back(normal);
54     }
55     else if (strcmp(lineHeader, "f") == 0) {
56         char line[256];
57         fgets(line, sizeof(line), file);
58
59         unsigned int vertexIndex[3], uvIndex[3], normalIndex[3];
60         int matches = sscanf(line, "%d/%d/%d %d/%d/%d %d/%d/%d",
61                             &vertexIndex[0], &uvIndex[0], &normalIndex[0],
62                             &vertexIndex[1], &uvIndex[1], &normalIndex[1],
63                             &vertexIndex[2], &uvIndex[2], &normalIndex[2]);
64
65         if (matches != 9) {
66             printf("File can't be read by our simple parser :-( Try exporting with other options\n");
67             fclose(file);
68             return false;
69         }
70
71         // Przypisanie materiału dla każdego trójkąta wewnętrznie
72         std::string materialName = currentMaterial;
73         Material material = materials[materialName];
74
75         // Get the attributes thanks to the index
76         glm::vec3 vertex[3];
77         glm::vec2 uv[3];
78         glm::vec3 normal[3];
79
80         for (int i = 0; i < 3; ++i) {
81             vertex[i] = temp_vertices[vertexIndex[i] - 1];
82             uv[i] = temp_uvns[uvIndex[i] - 1];
83             normal[i] = temp_normals[normalIndex[i] - 1];
84
85             // Put the attributes in buffers
86             out_vertices.push_back(vertex[i]);
87             out_uvns.push_back(uv[i]);
88             out_normals.push_back(normal[i]);
89             out_materials.push_back(material);
90         }
91     }
92     else if (strcmp(lineHeader, "usemtl") == 0) {
93         fscanf(file, "%s", lineHeader);
94         currentMaterial = lineHeader;
95     }
96     else {
97         char stupidBuffer[1000];
98         fgets(stupidBuffer, 1000, file);
99     }
100 }
101 fclose(file);
102 return true;
103 }

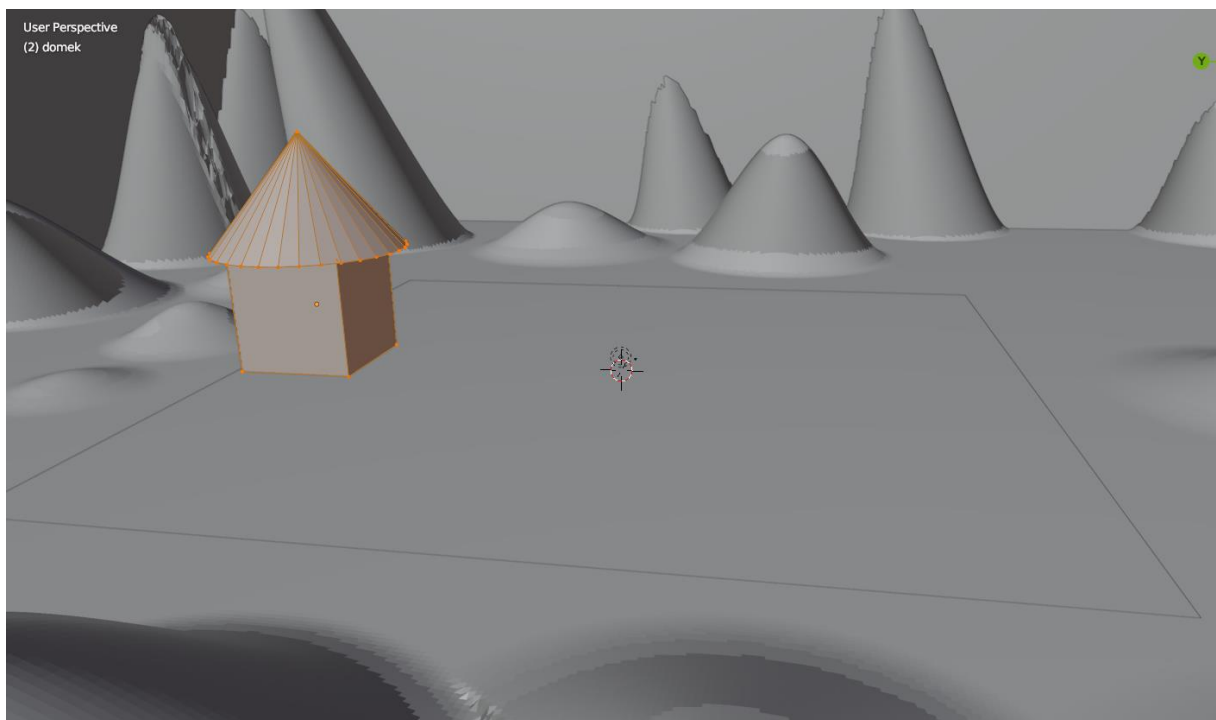
```

Funkcja, z której korzystamy wczytuje linia po linii z pliku .obj wygenerowanego z Blendera. Dzięki temu wczytujemy po kolei wierzchołki, normalne, maski. Dodatkowo mamy tutaj wczytanie .mtl z którego przypisujemy jakie tekstury są nam potrzebne.

- Instrukcja IV

Wykonując kolejną instrukcję stworzymy również w Blenderze trzy grupy otoczenia: trawę, wyżyny i góry. Wyżyny są zróżnicowane pod względem terenu i jego wysokości. Dodatkowo importujemy prosty domek jako nasz obiekt do kolizji. W tym fragmencie wczytujemy wszystkie potrzebne nam obiekty jako pliki obj.

```
std::vector<glm::vec3> vertices_trawa, vertices_wyzyny, vertices_gory, vertices_domek, vertices_lazik, vertices_kola, vertices_spiral, vertices_kret;  
std::vector<glm::vec2> uvs_trawa, uvs_wyzyny, uvs_gory, uvs_domek, uvs_lazik, uvs_kola, uvs_spiral, uvs_kret;  
std::vector<glm::vec3> normals_trawa, normals_wyzyny, normals_gory, normals_domek, normals_lazik, normals_kola, normals_spiral, normals_kret;  
std::vector<Material> materials_trawa, materials_wyzyny, materials_gory, materials_domek, materials_lazik, materials_kola, materials_spiral, materials_kret;  
//Wczytaj spiral  
bool res_spiral = loadOBJ("OBJ/spiral_export.obj", vertices_spiral, uvs_spiral, normals_spiral, materials_spiral, "OBJ/spiral_export.mtl");  
// Wczytaj gory  
bool res_gory = loadOBJ("OBJ/gory.obj", vertices_gory, uvs_gory, normals_gory, materials_gory, "OBJ/gory.mtl");  
//Wczytaj kola  
bool res_kola = loadOBJ("OBJ/kola3000.obj", vertices_kola, uvs_kola, normals_kola, materials_kola, "OBJ/kola3000.mtl");  
// Wczytaj srodek  
bool res_trawa = loadOBJ("OBJ/srodek.obj", vertices_trawa, uvs_trawa, normals_trawa, materials_trawa, "OBJ/srodek.mtl");  
// Wczytaj wyzyny  
bool res_wyzyny = loadOBJ("OBJ/wyzyny.obj", vertices_wyzyny, uvs_wyzyny, normals_wyzyny, materials_wyzyny, "OBJ/wyzyny.mtl");  
// Wczytaj lazik  
bool res_lazik = loadOBJ("OBJ/lazik3.obj", vertices_lazik, uvs_lazik, normals_lazik, materials_lazik, "OBJ/lazik3.mtl");  
//Wczytaj domek  
bool res_domek = loadOBJ("OBJ/domek.obj", vertices_domek, uvs_domek, normals_domek, materials_domek, "OBJ/domek.mtl");  
//wczytaj kret  
bool res_kret = loadOBJ("OBJ/kret.obj", vertices_kret, uvs_kret, normals_kret, materials_kret, "OBJ/kret.mtl");
```



Rysunek 1 Wygląd terenu w programie Blender

- Instrukcja V

Przy pomocy Blendera teksturujemy wszystko aby mniej więcej przypominało realistyczny teren. Nasz „łazik” również cały jest w teksturze.

```
// Wczytaj tekstury
std::map<std::string, GLuint> textureMap;
textureMap["grass"] = loadPNG("srodek_trawa.png");
textureMap["gory"] = loadPNG("gorytlo_texture.png");
textureMap["wyzyny"] = loadPNG("wyzyny_texture.png");
textureMap["łazik"] = loadPNG("main_base_color.png");
textureMap["domek"] = loadPNG("domek_textura.png");
textureMap["kola"] = loadPNG("kolo_Base_Color.png");
textureMap["spiral"] = loadPNG("spiral_Base_Color.png");
textureMap["kret"] = loadPNG("kret.png");
```

Udało się nam utworzyć mobilną kamerę, która obserwuje łazik. Dodatkowo możemy sterować nią przesuważąc myszką.

```
void computeMatricesFromInputs(GLFWwindow* window, glm::vec3 position, float angle){

    static double lastTime = glfwGetTime();
    double currentTime = glfwGetTime();
    float deltaTime = float(currentTime - lastTime);

    // Get mouse position
    double xpos, ypos;
    glfwGetCursorPos(window, &xpos, &ypos);

    // Reset mouse position for next frame
    glfwSetCursorPos(window, 1024/2, 768/2);

    // Compute new orientation
    horizontalAngle += mouseSpeed * float(1024/2 - xpos );
    verticalAngle += mouseSpeed * float( 768/2 - ypos );

    const float maxVerticalAngle = 3.14f; // Adjust this value as needed
    verticalAngle = glm::clamp(verticalAngle, -maxVerticalAngle, maxVerticalAngle);

    // Sprawdź, czy mysz się poruszyła
    if (xpos != lastMousePosX || ypos != lastMousePosY) {
        isMouseControlActive = true;
        mouseIdleTime = 0.0f; // resetuj czas bezczynności myszy

        // Oblicz nową orientację
        horizontalAngle += mouseSpeed * float(lastMousePosX - xpos);
        verticalAngle += mouseSpeed * float(lastMousePosY - ypos);

        // Ograniczenie zakresu obrotu kamery
        verticalAngle = glm::clamp(verticalAngle, -glm::radians(45.0f), glm::radians(45.0f));
    }
}
```

```

        // Aktualizuj ostatnią pozycję myszy
        lastMousePosX = xpos;
        lastMousePosY = ypos;
    }
    else {
        mouseIdleTime += deltaTime; // aktualizuj czas bezczynności myszy
    }

    // Resetuj orientację kamery, jeśli mysz była bezczynna przez określony czas
    if (isMouseControlActive && mouseIdleTime > mouseIdleTimeThreshold) {
        isMouseControlActive = false;
        horizontalAngle = 1.2f; // resetuj kąt poziomy
        verticalAngle = 1.45f; // resetuj kąt pionowy
    }

    glm::vec3 direction(cos(verticalAngle) * sin(horizontalAngle), sin(verticalAngle), cos(verticalAngle) * cos(horizontalAngle));

    //Right vector
    glm::vec3 right = glm::vec3(
        sin(horizontalAngle - 3.14f/2.0f),
        0,
        cos(horizontalAngle - 3.14f/2.0f)
    );

    // Up vector
    glm::vec3 up = glm::cross( right, direction );

    glm::vec3 cameraOffset = glm::vec3(10, 30, -20); // Zwiększenie wysokości kamery
    glm::vec3 cameraPosition;

    if (isMouseControlActive) {
        // Obliczenia pozycji kamery przy sterowaniu myszą
        glm::vec3 direction(
            cos(verticalAngle) * sin(horizontalAngle),
            sin(verticalAngle),
            cos(verticalAngle) * cos(horizontalAngle)
        );

        cameraPosition = position - direction * glm::length(cameraOffset) + glm::vec3(0, cameraOffset.y, 0);
    }
    else {
        // Kamera śledzi łożik
        glm::vec3 cameraDirection = glm::vec3(sin(glm::radians(angle)), 0, cos(glm::radians(angle)));
        cameraPosition = position - cameraDirection * glm::length(cameraOffset) + glm::vec3(0, cameraOffset.y, 0);
    }

    ViewMatrix = glm::lookAt(cameraPosition, position, glm::vec3(0, 1, 0));

    float FoV = initialFoV;
    float RenderDistance = 700.0f;

    // Projection matrix : 45° Field of View, 4:3 ratio, display range : 0.1 unit <-> 100 units
    ProjectionMatrix = glm::perspective(glm::radians(FoV), 4.0f / 3.0f, 0.1f, RenderDistance);

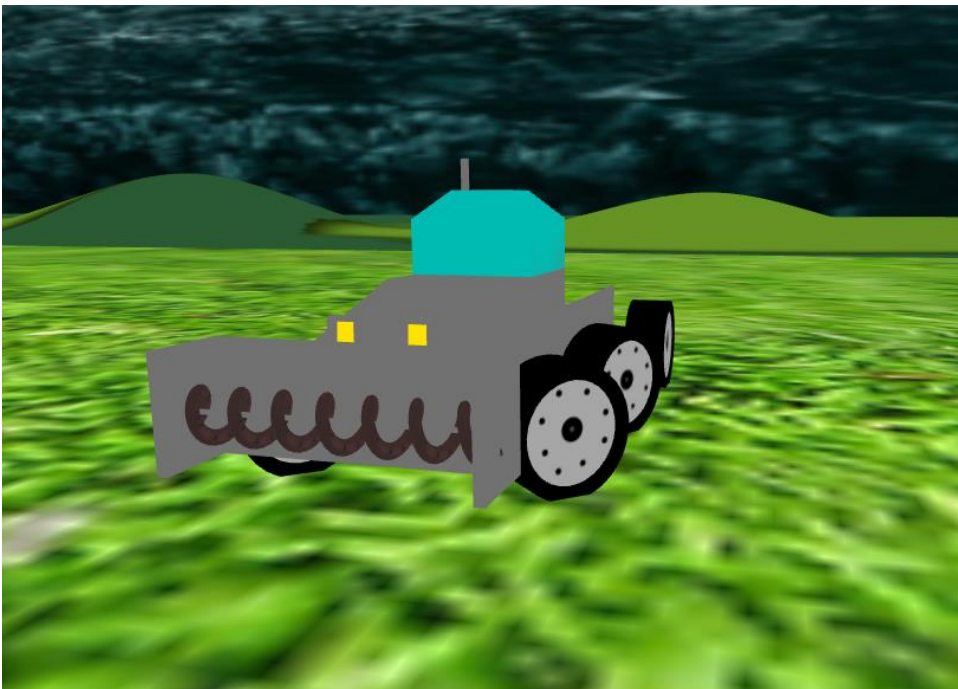
    lastTime = currentTime;
}

```

Ta funkcja jest dosyć rozbudowana, sprawdza czy sterujemy kamerą za pomocą myszki i zmienia nam konty według niej. Podczas dłuższej nie aktywności kamera wraca na ustaloną pozycję. Kamera aktualizuje swoją pozycję przypisując pozycję łożika i dodając offset-odległość od łożika, działanie to wykonuje się po komentarzu „//Kamera śledzi łożik”.

- Instrukcja VI

W łożyku zainstalowaliśmy system sterowania dwiema przednimi osiami. Ponad to łożyk ma napęd na wszystkie koła (6). Jeździ do przodu do tyłu i skręca. Żeby „wbić” wsteczny trzeba wcisnąć klawisz „2”, zaś do przodu „1” – jak podczas prowadzenia prawdziwego pojazdu. Klawisz W to gaz a S to hamulec. Klawiszami A i D sterujemy kierunkiem jazdy. Dodatkowo nasz łożyk posiada spiralę która się kreci, włączamy ją klawiszem R a wyłączamy T. Wykorzystujemy systemowy czas aby symulować stopniowe przyspieszanie i zwalnianie. Dotyczy to również kół które kręcą się w zależności od prędkości ale również po skręceniu wracają na swoją pierwotną pozycję.



```
for (KoloLazik kolo : kola_dane) {
    // Ustawianie macierzy transformacji dla srodka
    glm::mat4 ModelMatrix_kola = ModelMatrix_łożyk;

    ModelMatrix_kola = glm::translate(ModelMatrix_kola, kolo.position);

    if (kolo.numer_kola == 1 || kolo.numer_kola == 2) {
        ModelMatrix_kola = glm::rotate(ModelMatrix_kola, angle_skręt * 2.7f, glm::vec3(0.0f, 1.0f, 0.0f));
    }

    if (kolo.numer_kola == 3 || kolo.numer_kola == 4) {
        ModelMatrix_kola = glm::rotate(ModelMatrix_kola, angle_skręt * 1.8f, glm::vec3(0.0f, 1.0f, 0.0f));
    }

    ModelMatrix_kola = glm::rotate(ModelMatrix_kola, glm::radians(predkosc_obrotu_kol * 3), glm::vec3(1.0f, 0.0f, 0.0f));

    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS || glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {
        predkosc_obrotu_kol += predkosc * time * 1000;
    }
    else {
        predkosc_obrotu_kol += predkosc * time * 1000;
    }

    glm::mat4 MVP_kola = ProjectionMatrix * ViewMatrix * ModelMatrix_kola;
```

W kodzie powyżej realizujemy skręt kół i ich prędkość. Druga oś ma inny współczynnik skrętu. Używamy funkcji `translate()` i `rotate()` z biblioteki GLM. Wektory jako ostatnie argumenty w tych funkcjach oznaczają w jakich płaszczyznach mają obracać się obiekty.


```

ModelMatrix_lazik = glm::translate(ModelMatrix_lazik, position_lazik);

if (fabs(predkosc) > predkosc_przy_ktorej_lazik_skreca) {
    if (bieg == 1) {
        ModelMatrix_lazik = glm::rotate(ModelMatrix_lazik, glm::radians(angle_skret * 0.5f), glm::vec3(0, 1, 0));
    }
    else if (bieg == 2) {
        ModelMatrix_lazik = glm::rotate(ModelMatrix_lazik, glm::radians(-angle_skret * 0.5f), glm::vec3(0, 1, 0));
    }
}

pozycja_pojazdu = ModelMatrix_lazik[3];
if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS) {
    bieg = 1;
}
if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS) {
    bieg = 2;
}

if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS) {
    angle_skret -= predkosc_skretu * time;
}
else if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS) {
    angle_skret += predkosc_skretu * time;
}
else {
    if (angle_skret > 0) {
        angle_skret -= sila_hamowania_skretu_bez_hamulca * time;
    }
    else if (angle_skret < 0) {
        angle_skret += sila_hamowania_skretu_bez_hamulca * time;
    }
}

angle_skret = glm::clamp(angle_skret, -0.3f, 0.3f);
predkosc = glm::clamp(predkosc, -0.3f, 0.3f);
position_lazik = glm::clamp(position_lazik, -0.3f, 0.3f);

if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS) {
    predkosc = 0.0f;
    position_lazik.z = 0.0f;
    angle_skret = 0.0f;
}

if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {
    if (bieg == 1) {
        predkosc += przyspieszenie * time;
        position_lazik.z += przyspieszenie * time;
    }
    else if (bieg == 2) {
        predkosc -= przyspieszenie * time;
        position_lazik.z -= przyspieszenie * time;
    }
}

else if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) {
    if (bieg == 1) {
        if (predkosc > 0.f) {
            predkosc -= hamowanie * time;
            position_lazik.z -= hamowanie * time;
        }
    }
    else if (bieg == 2) {
        if (predkosc < 0.f) {
            predkosc += hamowanie * time;
            position_lazik.z += hamowanie * time;
        }
    }
}
}

```



```

else {
    if (fabs(predkosc) < 0.01f) {
        predkosc = 0.0f;
        position_lazik.z = 0.0f;
    }
    else if (predkosc > 0.f) {
        predkosc -= swobodne_hamowanie * time;
        position_lazik.z -= swobodne_hamowanie * time;
    }
    else if (predkosc < 0.f) {
        predkosc += swobodne_hamowanie * time;
        position_lazik.z += swobodne_hamowanie * time;
    }
}

glm::mat4 MVP_lazik = ProjectionMatrix * ViewMatrix * ModelMatrix_lazik;

```

Tutaj wykonuje się całe sterowanie. Zmieniamy położenie, prędkość, kąt skręcania. W razie nieoddziaływania łazik sam zatrzyma się pchany swoim pędem. Używamy tutaj funkcji clamp() aby ograniczyć nasze wartości, przecież nie można mieć prędkości dążącej do nieskończoności.

- Instrukcja VII

Tworzymy tutaj kolizję z domkiem i krętówką, o której nie wspominałem wcześniej. Używamy napisanej funkcji do sprawdzenia kolizji.

```

bool checkCollision(const glm::vec2& pos1, float radius1, const glm::vec2& pos2, float radius2) {
    float odleglosc = sqrt(pow((pos2.x - pos1.x), 2) + pow((pos2.y - pos1.y), 2));
    return odleglosc < radius1+radius2;
}

```

Obliczamy odległości pomiędzy środkami i sprawdzamy czy koła nachodzą na siebie.

```

if (checkCollision(glm::vec2(ModelMatrix_domek[3].x, ModelMatrix_domek[3].z), 45.f,
    glm::vec2(pozycja_pojazdu.x - 1.f, pozycja_pojazdu.z), 2.f)) {
    if (predkosc > 0.00000f) {
        predkosc -= 0.2f;
        position_lazik.z -= 0.2f;
    }
    else {
        predkosc += 0.2f;
        position_lazik.z += 0.2f;
    }
    angle_skret = 0.0f;
}

```

Powyżej dodajemy również fajną funkcjonalność – odbicie od przeszkody – gdy najedziemy na przeszkodę ta odbije nas w przeciwną stronę. W ten sposób unikamy problemu zablokowania.

Dosłownie to samo używamy aby rozliczyć się z krętówką, lecz tam gdy włączymy naszą spiralkę (klawisz R) i najedziemy przodem łazika (tyłem odbijemy się) to

krętówka zostanie posprzątana.

```
if (checkCollision(glm::vec2(ModelMatrix_kret[3].x, ModelMatrix_kret[3].z), 15.f,
    glm::vec2(pozycja_pojazdu.x - 1.f, pozycja_pojazdu.z), 2.f)) {
    if (bieg == 1 && licznik_R == 1) {
        losuj_pozycje_kretu(pozycja_kretu);
    }
    else {
        if (predkosc > 0.00000f) {
            predkosc -= 0.2f;
            position_lazik.z -= 0.2f;
        }
        else {
            predkosc += 0.2f;
            position_lazik.z += 0.2f;
        }
        angle_skret = 0.0f;
    }
}
```

- Instrukcja VIII

Ostatnia instrukcja ściśle powiązana jest z wcześniejszą, gdyż po wyczyszczeniu krętówki, sprytny kret tworzy kolejną w innej lokalizacji. Nie chcemy aby nasz ogród przed domem wyglądał jak plac budowy dlatego korzystamy z łazika i próbujemy go wysprzątać. Może kret kiedyś przestanie kopać???

```
void losuj_pozycje_kretu(glm::vec3& pos) {
    std::srand(time(NULL));
    float x = rand() % 200 + (-200);
    float y = rand() % 200 + (-200);
    if (checkCollision(glm::vec2(pos.x, pos.z), 10, glm::vec2(x, y), 10)) {
        x += 20.f;
        y += 20.f;
    }
    pos.x = x;
    pos.z = y;
}
```

